

Foundations of Spatial AI

Lecture 01 — Image Formation

Luca Di Giammarino, Emanuele Giacomini
Sapienza University of Rome

Compiled notes – September 23, 2026

Key idea

A photograph is a scene that has already been thrown away. This lecture is the *forward model*: how geometry, light and a sensor turn a room into pixels. Every week after it runs that model backwards. The camera matrix K introduced here is the one every reconstruction/rendering assumes you already know.

Contents

1	Agenda	2
2	Geometric primitives and transformations	3
2.1	2D points	3
2.1.1	Converting between coordinates	3
2.2	2D lines	4
2.3	Cross product	4
2.4	2D line arithmetic	4
2.5	2D conics	6
2.6	3D points	6
2.7	3D planes	6
2.8	3D lines	6
2.9	3D quadrics	7
2.10	2D transformations	7
2.10.1	Translation (2 DoF)	8
2.10.2	Euclidean / Rigid (3 DoF)	8
2.10.3	Similarity (4 DoF)	8
2.10.4	Affine (6 DoF)	8
2.10.5	Projective / Homography (8 DoF)	8
2.10.6	Transforming lines	9
2.11	3D transformations	9
2.12	Direct Linear Transform (DLT) for homography estimation	9
2.12.1	Solving the DLT with SVD	10
3	Geometric image formation	10
3.1	Origins of the pinhole camera	10
3.2	Physical vs. mathematical camera model	10
3.3	Projection models: orthographic vs. perspective	11
3.3.1	Focal length and camera distance	11

3.4	Orthographic projection	11
3.4.1	Matrix form	11
3.4.2	Scaled orthographic projection	12
3.5	Perspective projection	12
3.5.1	Matrix form	12
3.6	Principal point offset	12
3.7	The calibration matrix	12
3.8	Chaining transformations	13
3.9	Full rank representation	13
3.10	Lens distortion	13
3.10.1	Barrel and pincushion distortion	13
4	Photometric image formation	13
4.1	From light rays to pixel intensities	14
4.2	The rendering equation	14
4.3	Diffuse and specular reflection	14
4.4	Fresnel effect	14
4.5	Global illumination	14
4.6	Why camera lenses?	15
4.6.1	Pinhole size and image blur	15
4.7	Optics: pinhole vs. lens	15
4.8	The thin lens model	15
4.8.1	Focus and the circle of confusion	15
4.9	Aperture	15
4.10	The f-number	16
4.11	Depth of field (DOF)	16
4.12	Chromatic aberration	16
4.13	Vignetting	16
5	The image sensing pipeline (from photons to a file)	16
5.1	Shutter	17
5.2	Sensor: CCD vs. CMOS	17
5.3	Color Filter Arrays (CFA) and the Bayer pattern	17
5.3.1	Why more green?	17
5.4	Spectral sensitivity	17
5.5	Color spaces	17
5.6	Gamma compression	18
5.7	Image compression	18
6	Looking ahead	18

1 Agenda

1. **Primitives and transformations** – points, lines, planes, conics, quadrics, and how transformations move them.
2. **Geometric image formation** – the pinhole camera, projection, intrinsics, and where lenses bend the model.
3. **Photometric image formation** – the rendering equation, BRDFs, and what a real lens does to the light.

4. **The image sensing pipeline** – shutter, sensor, color filter array, gamma, compression: photons to a file.

2 Geometric primitives and transformations

Reference: Szeliski, Ch. 2.1; for a more exhaustive introduction: Hartley & Zisserman, *Multiple View Geometry in Computer Vision*.

Geometric primitives are the basic building blocks used to describe 3D shapes: points, lines and planes. The most basic transformations acting on them are also introduced.

2.1 2D points

A 2D point can be written in **inhomogeneous coordinates**:

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^2$$

or in **homogeneous coordinates**:

$$\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} \in \mathbb{P}^2, \quad \mathbb{P}^2 = \mathbb{R}^3 \setminus \{(0, 0, 0)\}$$

where $\mathbb{P}^2$ is called **projective space**.

Remark

Homogeneous vectors that differ only by scale are considered equivalent and define an equivalence class $\Rightarrow$ homogeneous vectors are defined only up to scale.

2.1.1 Converting between coordinates

From inhomogeneous to homogeneous (**augmented** vector $\bar{\mathbf{x}}$):

$$\tilde{\mathbf{x}} = \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix} = \bar{\mathbf{x}}$$

From homogeneous to inhomogeneous (divide by $\tilde{w}$):

$$\bar{\mathbf{x}} = \begin{pmatrix} \mathbf{x} \\ 1 \end{pmatrix}, \quad \tilde{\mathbf{x}} = \frac{1}{\tilde{w}} \begin{pmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} \tilde{x}/\tilde{w} \\ \tilde{y}/\tilde{w} \\ 1 \end{pmatrix}$$

Remark

Homogeneous points whose last element is $\tilde{w} = 0$ are called **ideal points** or **points at infinity**. These points cannot be represented with inhomogeneous coordinates.

The projective plane: geometrically, a homogeneous vector $\tilde{\mathbf{x}}$ is a ray in 3D space starting at the origin; the augmented vector $\bar{\mathbf{x}}$ is the intersection of this ray with the plane $\tilde{w} = 1$.

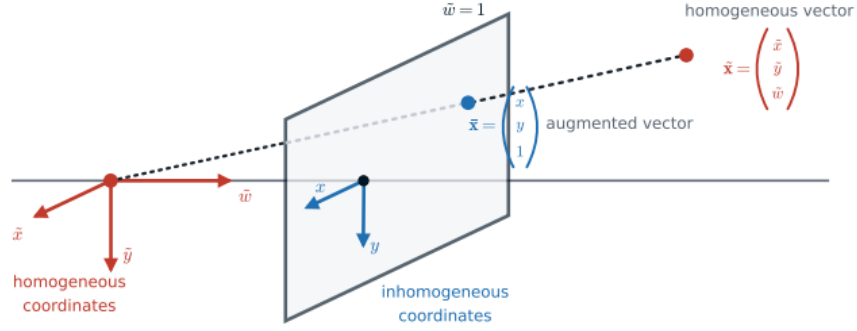


Figure 1: The projective plane

2.2 2D lines

2D lines are expressed using homogeneous coordinates $\tilde{l} = (a, b, c)^T$:

$$\{\bar{\mathbf{x}} \mid \tilde{l}^T \bar{\mathbf{x}} = 0\} \Leftrightarrow \{x, y \mid ax + by + c = 0\}$$

We can normalize $\tilde{l}$ so that $\tilde{l} = (n_x, n_y, -d)^T = (\mathbf{n}, -d)^T$ with $\|\mathbf{n}\|_2 = 1$: in this case $\mathbf{n}$ is the normal vector perpendicular to the line and d is its distance to the origin.

Remark

An exception is the **line at infinity** $\tilde{l}_\infty = (0, 0, 1)^T$, which passes through all ideal points.

2.3 Cross product

The cross product expressed as the product of a skew-symmetric matrix and a vector:

$$\mathbf{a} \times \mathbf{b} = [\mathbf{a}]_{\times} \mathbf{b} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

Remark

In this course, squared brackets $[\cdot]$ are used to distinguish matrices from vectors.

2.4 2D line arithmetic

In homogeneous coordinates:

- The **intersection** of two lines: $\tilde{\mathbf{x}} = \tilde{l}_1 \times \tilde{l}_2$
- The **line joining** two points: $\tilde{l} = \bar{\mathbf{x}}_1 \times \bar{\mathbf{x}}_2$

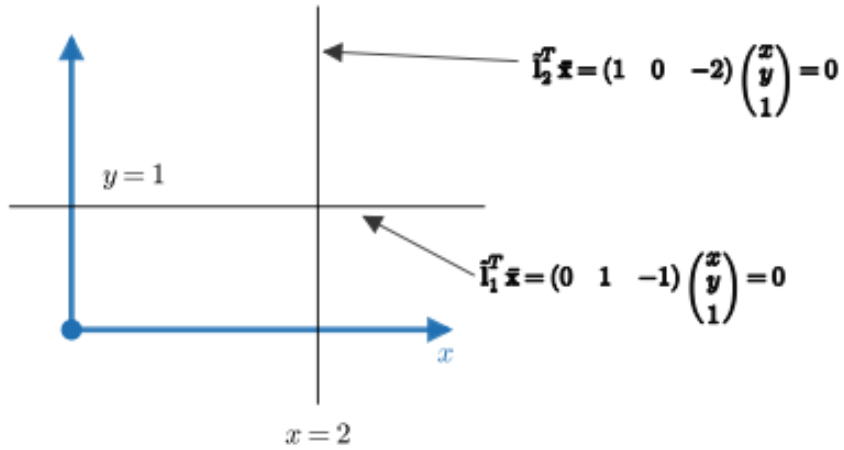


Figure 2: Intersection of two lines

Example: Intersection of two lines

Given the lines $y = 1$ ($\tilde{l}_1 = (0, 1, -1)^T$) and $x = 2$ ($\tilde{l}_2 = (1, 0, -2)^T$):

$$\tilde{l}_1 \times \tilde{l}_2 = \begin{bmatrix} 0 & 1 & 1 \\ -1 & 0 & 0 \\ -1 & 0 & 0 \end{bmatrix} \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix} = \begin{pmatrix} -2 \\ -1 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

i.e. the point $(2, 1)$, as expected from the geometric intersection.

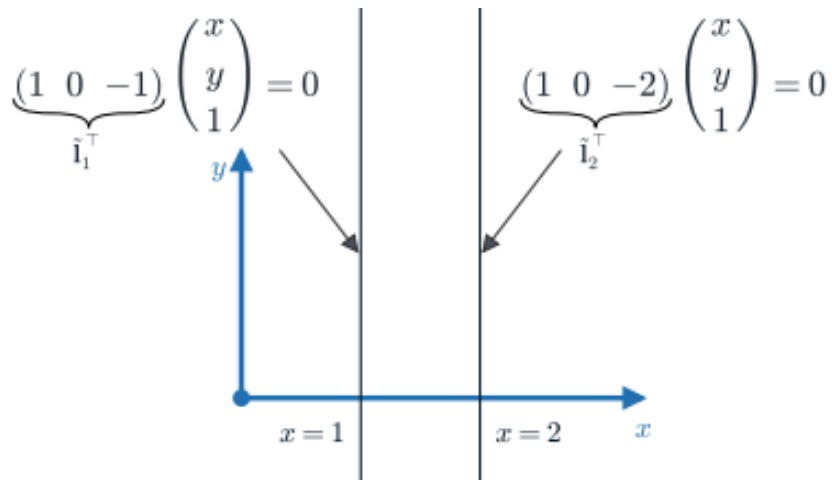


Figure 3: Parallel lines meet at infinity

Example: Parallel lines meet at infinity

For $x = 1$ and $x = 2$ (parallel lines), $\tilde{l}_1 \times \tilde{l}_2 = (0, 1, 0)^T$, i.e. a point with $\tilde{w} = 0$: an ideal point. Indeed $\tilde{l}_\infty^T(0, 1, 0)^T = 0$, confirming that this point lies on the line at infinity.

2.5 2D conics

More complex algebraic objects can be represented using polynomial homogeneous equations. **Conic sections** (arising as the intersection of a plane and a 3D cone) can be written using quadric equations:

$$\{\bar{\mathbf{x}} \mid \bar{\mathbf{x}}^T \mathbf{Q} \bar{\mathbf{x}} = 0\}$$

Types: circle, ellipse, parabola, hyperbola. Useful for multi-view geometry and camera calibration.

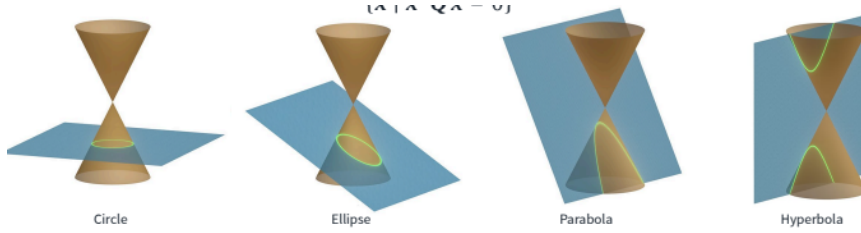


Figure 4: 2D Conics

2.6 3D points

Inhomogeneous coordinates: $\mathbf{x} = (x, y, z)^T \in \mathbb{R}^3$. Homogeneous coordinates: $\tilde{\mathbf{x}} = (\tilde{x}, \tilde{y}, \tilde{z}, \tilde{w})^T \in \mathbb{P}^3$, with $\mathbb{P}^3 = \mathbb{R}^4 \setminus \{(0, 0, 0, 0)\}$.

2.7 3D planes

3D planes can be represented as homogeneous coordinates $\tilde{\mathbf{m}} = (a, b, c, d)^T$:

$$\{\bar{\mathbf{x}} \mid \tilde{\mathbf{m}}^T \bar{\mathbf{x}} = 0\} \Leftrightarrow \{x, y, z \mid ax + by + cz + d = 0\}$$

Again, we can normalize $\tilde{\mathbf{m}}$ so that $\tilde{\mathbf{m}} = (n_x, n_y, n_z, -d)^T = (\mathbf{n}, -d)^T$ with $\|\mathbf{n}\|_2 = 1$: $\mathbf{n}$ is the normal perpendicular to the plane, d is its distance to the origin.

Remark

An exception is the **plane at infinity** $\tilde{\mathbf{m}} = (0, 0, 0, 1)^T$, which passes through all ideal points (= points at infinity) for which $\tilde{w} = 0$.

2.8 3D lines

3D lines are less elegant than either 2D lines or 3D planes. One possible representation is to express points on a line as a linear combination of two points $\mathbf{p}$ and $\mathbf{q}$ on the line:

$$\{\mathbf{x} \mid \mathbf{x} = (1 - \lambda)\mathbf{p} + \lambda\mathbf{q}, \lambda \in \mathbb{R}\}$$

However, this representation uses 6 parameters for 4 degrees of freedom. Alternative minimal representations are the **two-plane parameterization** or **Plücker coordinates** (see Szeliski Ch. 2.1, Hartley/Zisserman Ch. 2 for details).

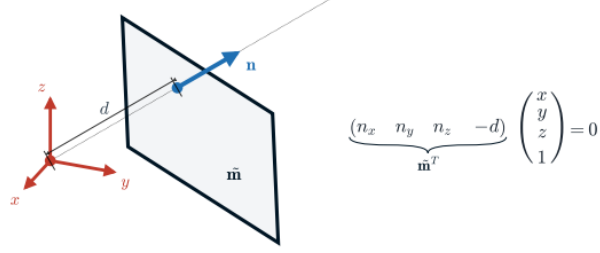


Figure 5: Plane normal and distance

2.9 3D quadrics

The 3D analog of 2D conics is a **quadric surface**:

$$\{\bar{\mathbf{x}} \mid \bar{\mathbf{x}}^T \mathbf{Q} \bar{\mathbf{x}} = 0\}$$

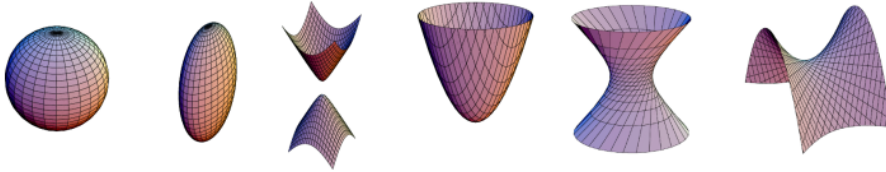


Figure 6: 3D Quadrics

Useful in the study of multi-view geometry. Also serves as a useful modeling primitive (spheres, ellipsoids, cylinders). **Superquadrics** (a generalization of quadrics) are used for shape abstraction and compression (e.g. Paschalidou, Ulusoy and Geiger: *Superquadrics Revisited*, CVPR 2019).



Figure 7: Superquadrics

2.10 2D transformations

There is a nested hierarchy of 2D transformations, each adding degrees of freedom (DoF) with respect to the previous one:

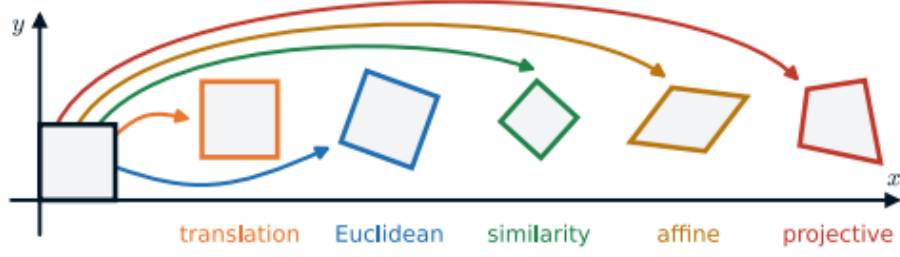


Figure 8: 2D Transformations

Transformation	Matrix (2×3)	DoF	Preserves
Translation	$[I \ \mathbf{t}]$	2	orientation
Rigid (Euclidean)	$[R \ \mathbf{t}]$	3	lengths
Similarity	$[sR \ \mathbf{t}]$	4	angles
Affine	$[A]$	6	parallelism
Projective	$[\tilde{H}]_{3 \times 3}$	8	straight lines

2.10.1 Translation (2 DoF)

$$\mathbf{x}' = \mathbf{x} + \mathbf{t} \quad \Leftrightarrow \quad \bar{\mathbf{x}}' = \begin{bmatrix} I & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \bar{\mathbf{x}}$$

2.10.2 Euclidean / Rigid (3 DoF)

$$\mathbf{x}' = R\mathbf{x} + \mathbf{t} \quad \Leftrightarrow \quad \bar{\mathbf{x}}' = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \bar{\mathbf{x}}$$

where $R \in SO(2)$ is an orthonormal rotation matrix with $RR^T = I$ and $\det(R) = 1$. Euclidean transformations preserve Euclidean distances.

2.10.3 Similarity (4 DoF)

$$\mathbf{x}' = sR\mathbf{x} + \mathbf{t} \quad \Leftrightarrow \quad \bar{\mathbf{x}}' = \begin{bmatrix} sR & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \bar{\mathbf{x}}$$

$R \in SO(2)$ is a rotation matrix and s is an arbitrary scale factor. The similarity transform preserves angles between lines.

2.10.4 Affine (6 DoF)

$$\mathbf{x}' = A\mathbf{x} + \mathbf{t} \quad \Leftrightarrow \quad \bar{\mathbf{x}}' = \begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \bar{\mathbf{x}}$$

with $A \in \mathbb{R}^{2 \times 2}$ an arbitrary 2×2 matrix. Parallel lines remain parallel under affine transformations.

2.10.5 Projective / Homography (8 DoF)

$$\tilde{\mathbf{x}}' = \tilde{H}\tilde{\mathbf{x}} \quad \left(\bar{\mathbf{x}} = \frac{1}{w}\tilde{\mathbf{x}} \right)$$

with $\tilde{H} \in \mathbb{R}^{3 \times 3}$ an arbitrary homogeneous matrix (specified up to scale). Projective transformations preserve straight lines.

Key idea

Transformations form a **nested set of groups** (closed under composition, inverse). They can be interpreted as restricted 3×3 matrices operating on 2D homogeneous coordinates.

2.10.6 Transforming lines

Considering any perspective 2D transformation $\tilde{\mathbf{x}}' = \tilde{H}\tilde{\mathbf{x}}$, the transformed 2D line equation is given by:

$$\tilde{l}'^T \tilde{\mathbf{x}}' = \tilde{l}'^T \tilde{H} \tilde{\mathbf{x}} = (\tilde{H}^T \tilde{l}')^T \tilde{\mathbf{x}} = \tilde{l}^T \tilde{\mathbf{x}} = 0$$

Therefore, we have:

$$\boxed{\tilde{l}' = \tilde{H}^{-T} \tilde{l}}$$

Thus, the action of a projective transformation on a dual object such as a 2D line (or a 3D normal) can be represented by the **transposed inverse** of the matrix.

2.11 3D transformations

3D transformations are defined analogously to 2D transformations; 3×4 matrices are extended with a fourth row $[0^T \ 1]$ for homogeneous transforms.

Transformation	Matrix (3×4)	DoF	Preserves
Translation	$[I \ \mathbf{t}]$	3	orientation
Rigid (Euclidean)	$[R \ \mathbf{t}]$	6	lengths
Similarity	$[sR \ \mathbf{t}]$	7	angles
Affine	$[A]$	12	parallelism
Projective	$[\tilde{H}]_{4 \times 4}$	15	straight lines

2.12 Direct Linear Transform (DLT) for homography estimation

Problem: estimate a homography from a set of 2D correspondences.

Let $\mathcal{X} = \{\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}'_i\}_{i=1}^N$ denote a set of N 2D-to-2D correspondences related by $\tilde{\mathbf{x}}'_i = \tilde{H}\tilde{\mathbf{x}}_i$. As the correspondence vectors are homogeneous, they have the same direction but differ in magnitude. Thus, the equation above can be expressed as:

$$\tilde{\mathbf{x}}'_i \times \tilde{H}\tilde{\mathbf{x}}_i = \mathbf{0}$$

Using $\tilde{\mathbf{h}}_k^T$ to denote the k -th row of $\tilde{H}$, this can be rewritten as a linear equation in $\tilde{\mathbf{h}}$:

$$\begin{bmatrix} \mathbf{0}^T & -\tilde{w}'_i \tilde{\mathbf{x}}_i^T & \tilde{y}'_i \tilde{\mathbf{x}}_i^T \\ \tilde{w}'_i \tilde{\mathbf{x}}_i^T & \mathbf{0}^T & -\tilde{x}'_i \tilde{\mathbf{x}}_i^T \\ -\tilde{y}'_i \tilde{\mathbf{x}}_i^T & \tilde{x}'_i \tilde{\mathbf{x}}_i^T & \mathbf{0}^T \end{bmatrix} \begin{bmatrix} \tilde{\mathbf{h}}_1 \\ \tilde{\mathbf{h}}_2 \\ \tilde{\mathbf{h}}_3 \end{bmatrix} = \mathbf{0}$$

The last row is linearly dependent (up to scale) on the first two and can be dropped: each correspondence yields **two equations**.

2.12.1 Solving the DLT with SVD

Stacking all equations into a $2N \times 9$ dimensional matrix A leads to the following constrained least squares problem:

$$\tilde{\mathbf{h}}^* = \underset{\tilde{\mathbf{h}}}{\operatorname{argmin}} \|\mathbf{A}\tilde{\mathbf{h}}\|_2^2 + \lambda \left(\|\tilde{\mathbf{h}}\|_2^2 - 1 \right) = \underset{\tilde{\mathbf{h}}}{\operatorname{argmin}} \tilde{\mathbf{h}}^T \mathbf{A}^T \mathbf{A} \tilde{\mathbf{h}} + \lambda \left(\tilde{\mathbf{h}}^T \tilde{\mathbf{h}} - 1 \right)$$

where we have fixed $\|\tilde{\mathbf{h}}\|_2^2 = 1$ as $\tilde{H}$ is homogeneous (i.e. defined only up to scale) and the trivial solution $\tilde{\mathbf{h}} = \mathbf{0}$ is not of interest.

Key idea

The solution is the **singular vector** corresponding to the smallest singular value of A (i.e. the last column of V when decomposing $A = UDV^T$). The resulting algorithm is called **Direct Linear Transformation (DLT)**.

Application: panorama stitching (Brown & Lowe, ICCV 2003) – homographies between overlapping images are estimated to fuse them into a single panoramic image.

3 Geometric image formation

The pinhole camera, perspective projection, and the calibration matrix that takes a point in the world to a pixel.

3.1 Origins of the pinhole camera

- **Animal eye:** a long time ago.
- **Pinhole perspective projection:** Brunelleschi, 15th century.
- **Photographic camera:** Nicéphore Niépce, 1816.
- **Camera obscura:** known as early as the 4th century BC.

The camera obscura is reproduced artistically today (e.g. Abelardo Morell) and even with tallboy beer cans or small sections of PVC pipe as a pinhole for months-long exposures.

3.2 Physical vs. mathematical camera model

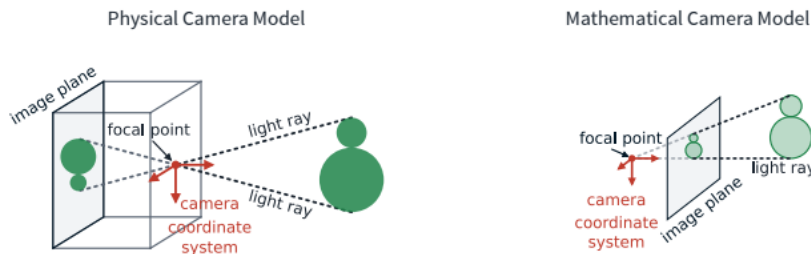


Figure 9: Camera Models

- In a **physical** pinhole camera the image is projected *upside-down* onto the image plane, which is located *behind* the focal point.

- In the **mathematical** camera model, when modeling perspective projection, we assume the image plane is *in front*.
- Both models are equivalent, with an appropriate change of image coordinates.

3.3 Projection models: orthographic vs. perspective

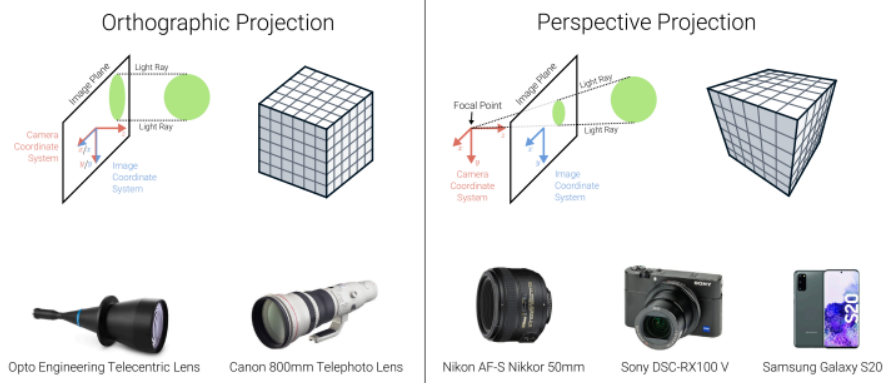


Figure 10: Projection Models

- **Orthographic projection:** light rays parallel to the optical axis (e.g. telecentric lenses, long telephoto lenses).
- **Perspective projection:** light rays converging at the focal point (e.g. normal camera and smartphone lenses).

These are the two most important projections (see Szeliski Ch. 2.1.4 for others).

3.3.1 Focal length and camera distance

As focal length (or distance from the camera) increases, perspective projection tends toward **weak perspective** / **orthographic** projection: the vanishing point moves further away and converging lines become increasingly parallel.

3.4 Orthographic projection

Orthographic projection of a 3D point $\mathbf{x}_c \in \mathbb{R}^3$ to pixel coordinates $\mathbf{x}_s \in \mathbb{R}^2$:

- The x and y axes of the camera and image coordinate systems are shared.
- Light rays are parallel to the z -coordinate of the camera coordinate system.
- During projection, the z -coordinate is dropped; x and y remain the same.

3.4.1 Matrix form

$$\mathbf{x}_s = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \mathbf{x}_c \quad \Leftrightarrow \quad \bar{\mathbf{x}}_s = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bar{\mathbf{x}}_c$$

Remark

Orthography is exact for telecentric lenses and an approximation for telephoto lenses. After projection the distance of the 3D point from the image can't be recovered.

3.4.2 Scaled orthographic projection

In practice, world coordinates (which may measure dimensions in meters) must be scaled to fit onto an image sensor (measuring in pixels) $\Rightarrow$ **scaled orthography**:

$$\mathbf{x}_s = \begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \end{bmatrix} \mathbf{x}_c \Leftrightarrow \bar{\mathbf{x}}_s = \begin{bmatrix} s & 0 & 0 & 0 \\ 0 & s & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \bar{\mathbf{x}}_c$$

The unit for s is px/m or px/mm . Under orthography, structure and motion can be estimated simultaneously using **factorization methods** (e.g. via singular value decomposition).

3.5 Perspective projection

Perspective projection of a 3D point $\mathbf{x}_c$ to pixel coordinates $\mathbf{x}_s$:

- The light ray passes through the camera center, the pixel $\mathbf{x}_s$ and the point $\mathbf{x}_c$.
- Convention: the principal axis (orthogonal to the image plane) aligns with the z -axis.

3.5.1 Matrix form

3D points in camera coordinates are mapped to the image plane by **dividing** them by their z component and multiplying with the focal length:

$$\begin{pmatrix} x_s \\ y_s \end{pmatrix} = \begin{pmatrix} f x_c / z_c \\ f y_c / z_c \end{pmatrix} \Leftrightarrow \tilde{\mathbf{x}}_s = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{\mathbf{x}}_c$$

Remark

This projection is **linear** when using homogeneous coordinates. After the projection it is not possible to recover the distance of the 3D point from the image. The unit for f is px (pixels).

3.6 Principal point offset

To ensure positive pixel coordinates, a **principal point offset** $\mathbf{c} = (c_x, c_y)^T$ is usually added: this moves the image coordinate system from the principal axis to the corner of the image plane.

3.7 The calibration matrix

The complete perspective projection model:

$$\begin{pmatrix} x_s \\ y_s \end{pmatrix} = \begin{pmatrix} f_x x_c / z_c + s y_c / z_c + c_x \\ f_y y_c / z_c + c_y \end{pmatrix} \Leftrightarrow \tilde{\mathbf{x}}_s = \begin{bmatrix} f_x & s & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \bar{\mathbf{x}}_c$$

- The left 3×3 submatrix of the projection matrix is called the **calibration matrix** K .
- The parameters of K are called camera **intrinsics** (as opposed to extrinsic pose).
- f_x and f_y are independent, allowing for different pixel aspect ratios.
- The **skew** s arises because the sensor is not mounted perpendicular to the optical axis.
- In practice, we often set $f_x = f_y$ and $s = 0$, but still model $\mathbf{c} = (c_x, c_y)^T$.

3.8 Chaining transformations

Let K be the calibration matrix (intrinsics) and $[R \mid \mathbf{t}]$ the camera pose (extrinsics). We chain both transformations to project a point in world coordinates to the image:

$$\tilde{\mathbf{x}}_s = [K \ 0] \tilde{\mathbf{x}}_c = [K \ 0] \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \tilde{\mathbf{x}}_w = K[R \ \mathbf{t}] \tilde{\mathbf{x}}_w = P \tilde{\mathbf{x}}_w$$

Remark

The 3×4 projection matrix P can be **pre-computed**.

3.9 Full rank representation

It is sometimes preferable to use a full rank 4×4 projection matrix:

$$\tilde{\mathbf{x}}_s = \begin{bmatrix} K & \mathbf{0} \\ \mathbf{0}^T & 1 \end{bmatrix} \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \tilde{\mathbf{x}}_w = \tilde{P} \tilde{\mathbf{x}}_w$$

Now, the homogeneous vector $\tilde{\mathbf{x}}_s$ is a 4D vector and must be normalized with respect to its 3rd entry to obtain inhomogeneous image pixels:

$$\bar{\mathbf{x}}_s = \tilde{\mathbf{x}}_s / z_s = (x_s / z_s, y_s / z_s, 1, 1 / z_s)^T$$

Remark

The 4th component of the inhomogeneous 4D vector is the **inverse depth**. If the inverse depth is known, a 3D point can be retrieved from its pixel coordinates via $\tilde{\mathbf{x}}_w = \tilde{P}^{-1} \tilde{\mathbf{x}}_s$, followed by normalization of $\tilde{\mathbf{x}}_w$ with respect to its 4th entry.

3.10 Lens distortion

The assumption of linear projection (straight lines remain straight) is violated in practice due to the properties of the camera lens, which introduces distortions. Let $x = x_c / z_c$, $y = y_c / z_c$, and $r^2 = x^2 + y^2$. The distorted point is obtained as:

$$\mathbf{x}' = \underbrace{(1 + \kappa_1 r^2 + \kappa_2 r^4)}_{\text{radial distortion}} \begin{pmatrix} x \\ y \end{pmatrix} + \underbrace{\begin{pmatrix} 2\kappa_3 xy + \kappa_4(r^2 + 2x^2) \\ 2\kappa_4 xy + \kappa_3(r^2 + 2y^2) \end{pmatrix}}_{\text{tangential distortion}}$$

$$\mathbf{x}_s = \begin{pmatrix} f_x x' + c_x \\ f_y y' + c_y \end{pmatrix}$$

Both **radial and tangential distortion** effects can be modeled relatively easily. Images can be **undistorted** such that the perspective projection model applies. More complex distortion models must be used for wide-angle lenses (e.g. fisheye).

3.10.1 Barrel and pincushion distortion

- **Barrel distortion** ($\kappa_1 < 0$): lines bulge outward.
- **Pincushion distortion** ($\kappa_1 > 0$): lines pinch inward.

4 Photometric image formation

Geometry determines where a point lands. This part determines how bright it is: the rendering equation, reflectance, and what a real lens does to the light.

4.1 From light rays to pixel intensities

- So far we have discussed how individual light rays travel through space.
- We now discuss how an image is formed in terms of **pixel intensities and colors**.
- Light is **emitted** by one or more light sources and **reflected** or **refracted** (once or multiple times) at surfaces of objects (or media) in the scene.

4.2 The rendering equation

Let $\mathbf{p} \in \mathbb{R}^3$ denote a 3D surface point, $\mathbf{v} \in \mathbb{R}^3$ the viewing direction and $\mathbf{s} \in \mathbb{R}^3$ the incoming light direction. The **rendering equation** describes how much of the light L_{in} with wavelength λ arriving at $\mathbf{p}$ is reflected into the viewing direction $\mathbf{v}$:

$$L_{\text{out}}(\mathbf{p}, \mathbf{v}, \lambda) = L_{\text{emit}}(\mathbf{p}, \mathbf{v}, \lambda) + \int_{\Omega} \text{BRDF}(\mathbf{p}, \mathbf{s}, \mathbf{v}, \lambda) \cdot L_{\text{in}}(\mathbf{p}, \mathbf{s}, \lambda) \cdot (-\mathbf{n}^T \mathbf{s}) d\mathbf{s}$$

- Ω is the unit hemisphere at normal $\mathbf{n}$.
- The **bidirectional reflectance distribution function** (BRDF) $\text{BRDF}(\mathbf{p}, \mathbf{s}, \mathbf{v}, \lambda)$ defines how light is reflected at an opaque surface.
- $L_{\text{emit}} > 0$ only for light-emitting surfaces.

4.3 Diffuse and specular reflection

Typical BRDFs have a **diffuse** and a **specular** component:

- The diffuse (=constant) component scatters light uniformly in all directions $\Rightarrow$ this leads to **shading**, i.e. smooth variation of intensity with respect to the surface normal.
- The specular component depends strongly on the outgoing light direction (metal, shiny plastic, etc). A limiting case is a **mirror** (ideal reflection, angle of incidence = angle of reflection).
- Combining diffuse + specular yields the typical realistic appearance (e.g. a sphere with a specular highlight overlaid on diffuse shading).

Real-world BRDFs can be very complex and **spatially varying** (e.g. silk, peach skin, fabric, leaves, peacock feathers, hair).

4.4 Fresnel effect

The amount of light reflected from a surface depends on the **viewing angle** (classic example: reflections on water changing intensity depending on angle).

4.5 Global illumination

- Modeling one light bounce is **insufficient** for rendering complex scenes.
- Light sources can be **shadowed by occluders** and rays can bounce multiple times.
- **Global illumination** techniques also take indirect illumination into account (e.g. light bouncing off colored walls indirectly tinting a character).

4.6 Why camera lenses?

- Large and very small pinholes result in **image blur** (averaging, diffraction respectively).
- Small pinholes require very **long shutter times** ($\Rightarrow$ motion blur).

4.6.1 Pinhole size and image blur

- **Large pinhole:** multiple light rays from the same source point hit different areas of the sensor $\Rightarrow$ blurred image (averaging blur).
- **Small pinhole:** averaging blur decreases, but **diffraction** at the edge of the hole blurs it again. There is therefore an optimal pinhole size.

4.7 Optics: pinhole vs. lens

- Cameras use one or multiple **lenses** to accumulate light on the sensor plane.
- Importantly, if a 3D point is **in focus**, all light rays arrive at the same 2D pixel.
- For many applications it suffices to model lens cameras with a **pinhole model**.
- However, to address **focus, vignetting and aberration** we need to explicitly model lenses.

4.8 The thin lens model

Using a spherical thin lens as an approximation: From similar triangles:

$$\frac{x_s}{z_s - f} = \frac{x_c}{f}, \quad \frac{x_s}{z_s} = \frac{x_c}{z_c}$$

we obtain the **thin lens equation**:

$$\boxed{\frac{1}{z_s} + \frac{1}{z_c} = \frac{1}{f}}$$

Properties: axis-parallel rays pass through the focal point; rays through the center keep their direction.

4.8.1 Focus and the circle of confusion

- The image is **in focus** if $\frac{1}{z_s} + \frac{1}{z_c} = \frac{1}{f}$.
- For $z_c \rightarrow \infty$ we obtain $z_s = f$ (a lens with focal length f approximates a pinhole at distance f).
- If the image plane is **out of focus**, a 3D point projects to the **circle of confusion** c (instead of a single sharp point).

4.9 Aperture

- To control the size of the circle of confusion, we change the lens **aperture**.
- An aperture is a hole or an opening through which light travels.
- The aperture limits the amount of light that can reach the image plane.
- Smaller apertures lead to **sharper, but more noisy** images (fewer photons captured).

4.10 The f-number

- The allowable depth variation that limits the circle of confusion c is called **depth of field**, and is a function of both the focus distance and the lens aperture.
- Typical DSLR lenses have depth-of-field indicators.
- The commonly displayed **f-number** is defined as:

$$N = \frac{f}{d} \quad (\text{often denoted as } f/N, \text{ e.g. } f/1.4)$$

where d is the aperture diameter, i.e. N is the lens focal length f divided by the aperture diameter d .

4.11 Depth of field (DOF)

Definition: the distance between the nearest and farthest objects that are acceptably sharp.

- Decreasing the aperture diameter (i.e. *increasing* the f-number) **increases** the depth of field.
- Numerical example: $f/1.4 \Rightarrow \text{DOF} = 0.8 \text{ cm}$; $f/4.0 \Rightarrow \text{DOF} = 2.2 \text{ cm}$; $f/22 \Rightarrow \text{DOF} = 12.4 \text{ cm}$.

4.12 Chromatic aberration

- The **index of refraction** for glass varies slightly as a function of wavelength.
- Thus, simple lenses suffer from **chromatic aberration**, which is the tendency for light of different colors to focus at slightly different distances (blur, color shift).
- To reduce chromatic and other kinds of aberrations, most photographic lenses are **compound lenses** made of different glass elements (with different coatings).
- **High-quality** lenses minimize blur/rainbow edges; **low-quality** lenses clearly show blur and colored fringing at high-contrast edges.

4.13 Vignetting

- **Vignetting** is the tendency for the brightness to fall off towards the image edge.
- It is a composition of two effects:
 - **Natural vignetting:** foreshortening of the object surface and the lens aperture.
 - **Mechanical vignetting:** the shaded part of the beam never reaches the image.
- Vignetting can be **calibrated** (i.e. undone) in post-processing.

5 The image sensing pipeline (from photons to a file)

Everything between the photons and the array returned by `imread`: shutter, sensor, color filter array, gamma and compression.

The image sensing pipeline can be divided into three stages:

1. Physical light transport in the camera lens/body.
2. Photon measurement and conversion on the sensor chip.
3. Image signal processing (ISP) and image compression.

5.1 Shutter

- A **focal plane shutter** is positioned just in front of the image sensor / film.
- Most digital cameras use a **combination** of mechanical and electronic shutter.
- The **shutter speed** (exposure time) controls how much light reaches the sensor.
- It determines if an image appears over-/underexposed, blurred or noisy.

5.2 Sensor: CCD vs. CMOS

- **CCD** (Charge-Coupled Device): moves charge from pixel to pixel and converts it to voltage at the output node.
- **CMOS** (Complementary Metal-Oxide-Semiconductor): converts charge to voltage *inside each pixel*; this is the current standard.
- Larger chips (full frame = 35 mm) are more photo-sensitive $\Rightarrow$ less noise.
- Anatomy of the active pixel sensor photodiode: microlens $\rightarrow$ color filter (e.g. red) $\rightarrow$ reset transistor $\rightarrow$ amplifier transistor $\rightarrow$ photodiode (silicon substrate) $\rightarrow$ potential well.

5.3 Color Filter Arrays (CFA) and the Bayer pattern

- To measure color, pixels are arranged in a **color array**, e.g. the **Bayer RGB pattern** (repeating motif: G-R / B-G, with twice as many green as red or blue sensors).
- Missing colors at each pixel are **interpolated** from the neighbors: this process is called **demosaicing**.

5.3.1 Why more green?

The Bayer pattern uses twice as many green sensors as red and blue because the **human eye** (and the typical spectral sensitivity of sensors) is most sensitive in the green band (relative sensitivity curve peaking around $\sim 550\text{--}560\text{ nm}$).

5.4 Spectral sensitivity

Each pixel **integrates the light spectrum** L according to its spectral sensitivity S :

$$R = \int L(\lambda) S_R(\lambda) d\lambda$$

The spectral response curves (for monochrome, red, green, blue Bayer channels) are provided by the camera manufacturer.

5.5 Color spaces

Various different color spaces have been developed and are used in practice, for example:

- **RGB** (R, G, B channels).
- **CIELAB** (L^*, a^*, b^*): L^* = lightness, a^* = green-red axis, b^* = blue-yellow axis.
- **HSV** (Hue, Saturation, Value).

5.6 Gamma compression

- Humans are more sensitive to intensity differences in **darker regions**.
- Therefore, it is beneficial to **nonlinearly transform** the intensities or colors prior to discretization (during capture), and to undo this transformation during loading (decoding).

5.7 Image compression

- Typically **luminance** is compressed with higher fidelity than **chrominance**.
- Often, 8×8 **pixel patch-based** discrete cosine or wavelet transforms are used.
- The **Discrete Cosine Transform (DCT)** is an approximation to PCA on natural images.
- The coefficients are **quantized** to integers that can be stored with **Huffman codes**.
- More recently, **deep network based** compression algorithms have been developed.

6 Looking ahead

This lecture modeled how a scene becomes an image. Lecture 02 covers how to represent the scene itself: points (point clouds), meshes, voxels, signed distance functions (SDFs), Gaussians (3D Gaussian Splatting).

These slides are adapted from the Computer Vision course of Uni Tuebingen, Prof. Andreas Geiger.